VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

BHUMIKA SHRINIVAS TELI (1BM24CS072)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by **BHUMIKA SHRINIVAS TELI (1BM24CS072)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Dr. Namratha M
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

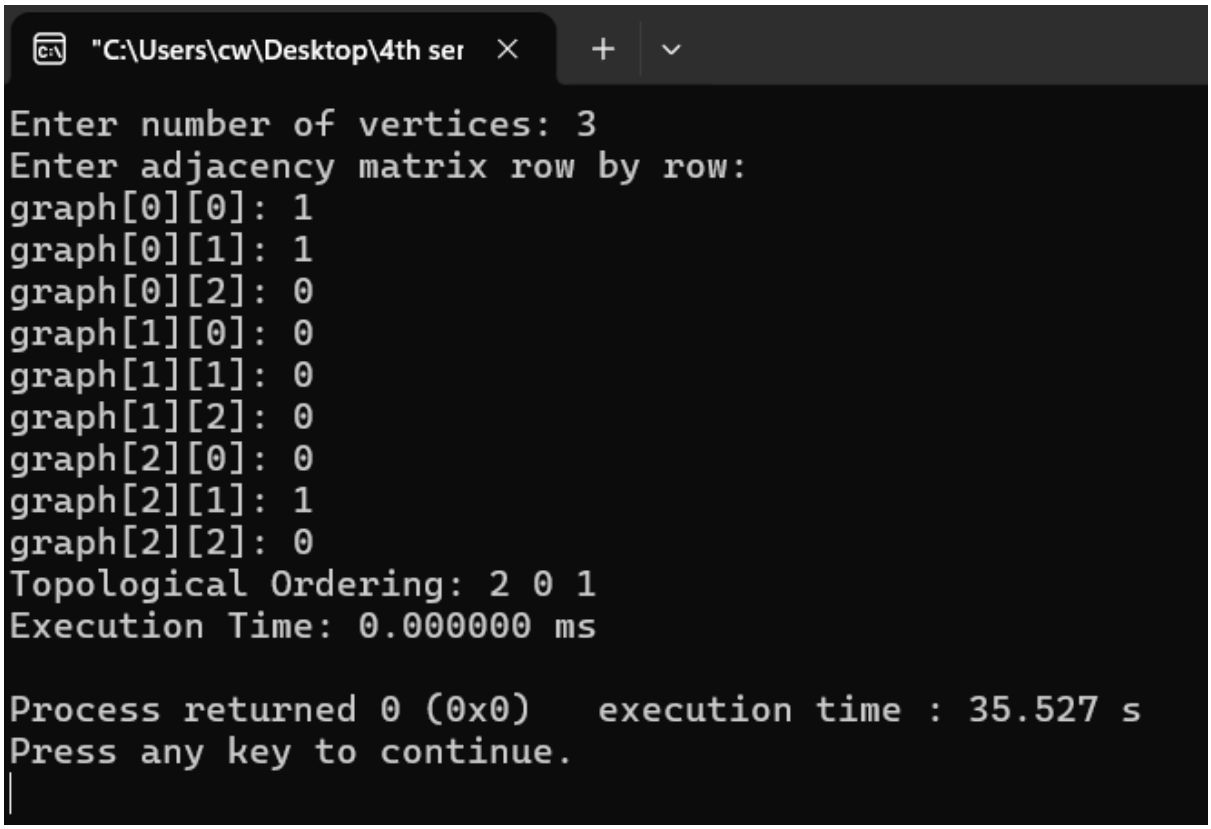
Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>
#include <time.h>
#define MAX 10
int graph[MAX][MAX], visited[MAX], stack[MAX];
int n, top = -1;
void dfs(int v)
{
   visited[v] = 1;
   for (int i = 0; i < n; i++)
   {
      if (graph[v][i] && !visited[i])
      {
         dfs(i);
      }
   }
   stack[++top] = v;
}
void topologicalSort()
{
   for (int i = 0; i < n; i++)
   {
      if (!visited[i])
      {
         dfs(i);
      }
```

```
    }
    printf("Topological Ordering: ");
    while (top != -1)
    {
        printf("%d ", stack[top--]);
    }
    printf("\n");
}
int main()
{
    clock_t start, end;
    double time_taken;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix row by row:\n");
    for (int i = 0; i < n; i++)
    {
        visited[i] = 0;
        for (int j = 0; j < n; j++)
        {
            printf("graph[%d][%d]: ", i, j);
            scanf("%d", &graph[i][j]);
        }
    }
    start = clock();
    topologicalSort();
    end = clock();
    time_taken = ((double)(end - start) * 1000) / CLOCKS_PER_SEC;
    printf("Execution Time: %lf ms\n", time_taken);
```

```
    return 0;
}
```

Output:



Lab Program 2:

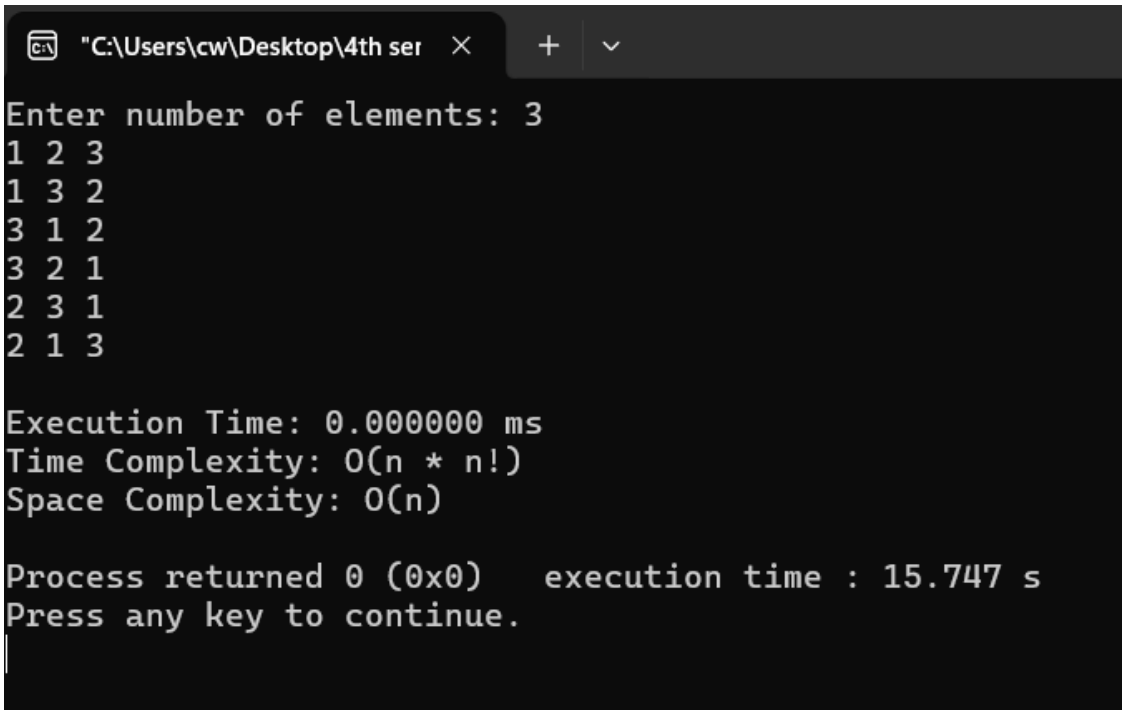
Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>
#include <time.h>
int main()
{
   int n;
   clock_t start, end;
   double time_taken;
   printf("Enter number of elements: ");
   scanf("%d", &n);
   int p[n], dir[n];
   for (int i = 0; i < n; i++)
   {
      p[i] = i + 1;
      dir[i] = -1;
   }
   start = clock();
   for (int i = 0; i < n; i++)
      printf("%d ", p[i]);
   printf("\n");
   while (1)
   {
      int mobile = 0, pos = -1;
      for (int i = 0; i < n; i++)
      {
         if (dir[p[i] - 1] == -1 &&
            i > 0 &&
            p[i] > p[i - 1] &&
```

```
            p[i] > mobile)
        {
            mobile = p[i];
            pos = i;
        }
        if (dir[p[i] - 1] == 1 &&
            i < n - 1 &&
            p[i] > p[i + 1] &&
            p[i] > mobile)
        {
            mobile = p[i];
            pos = i;
        }
    }
    if (mobile == 0)
        break;
    if (dir[mobile - 1] == -1)
    {
        int temp = p[pos];
        p[pos] = p[pos - 1];
        p[pos - 1] = temp;
        pos--;
    }
    else
    {
        int temp = p[pos];
        p[pos] = p[pos + 1];
        p[pos + 1] = temp;
        pos++;
```

```
        }
        for (int i = 0; i < n; i++)
        {
            if (p[i] > mobile)
                dir[p[i] - 1] *= -1;
        }
        for (int i = 0; i < n; i++)
            printf("%d ", p[i]);
        printf("\n");
    }
    end = clock();
    time_taken = ((double)(end - start) * 1000) / CLOCKS_PER_SEC;
    printf("\nExecution Time: %lf ms\n", time_taken);
    printf("Time Complexity: O(n * n!)\n");
    printf("Space Complexity: O(n)\n");
    return 0;
}
```

Output:



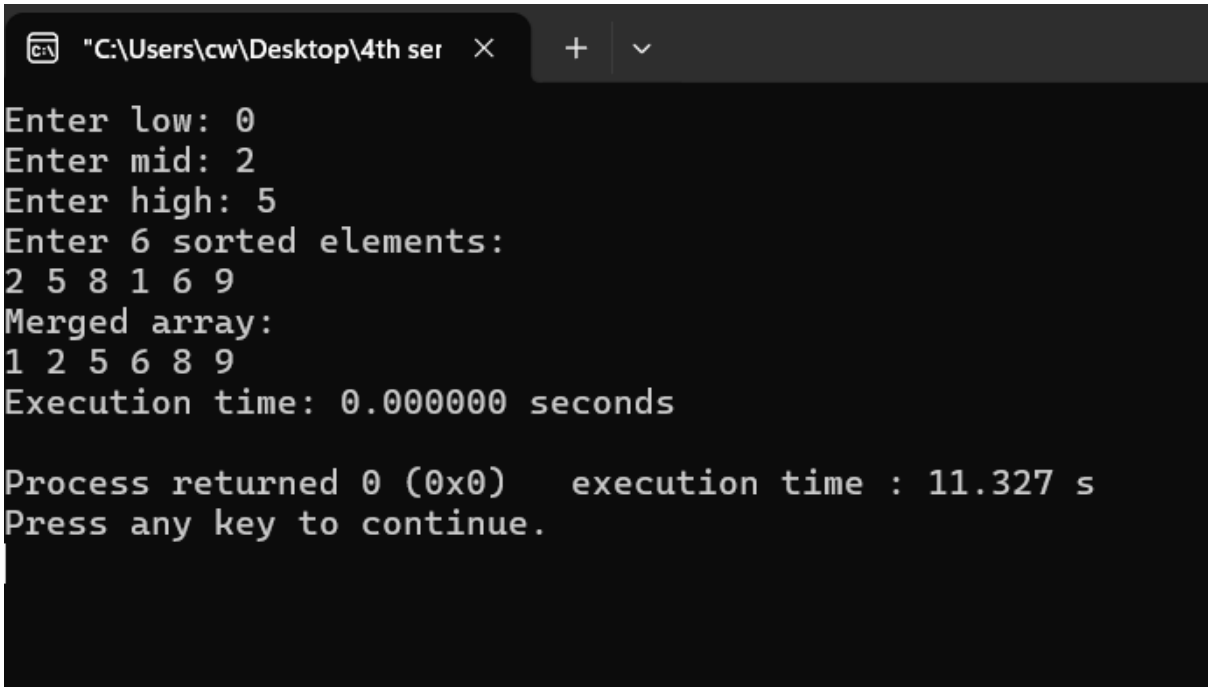
Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include <stdio.h>
#include <time.h>
int main(){
   int low, mid, high;
   printf("Enter low: ");
   scanf("%d",&low);
   printf("Enter mid: ");
   scanf("%d",&mid);
   printf("Enter high: ");
   scanf("%d",&high);
   int a[100], b[100];
   printf("Enter %d sorted elements:\n", high+1);
   for(int i=low;i<=high;i++){
      scanf("%d",&a[i]);
   }
   clock_t start, end;
   double cpu_time_used;
   start = clock();  // start time
   int i = low;
   int j = mid + 1;
   int k = low;
   while(i<=mid && j<=high){
      if(a[i] <= a[j]){
         b[k] = a[i];
         i++;
      }
      else{
         b[k] = a[j];
```

```
            j++;
        }
        k++;
    }
    while(i<=mid){
        b[k] = a[i];
        i++;
        k++;
    }
    while(j<=high){
        b[k] = a[j];
        j++;
        k++;
    }
    end = clock();  // end time
    cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
    printf("Merged array:\n");
    for(k=low;k<=high;k++){
        printf("%d ", b[k]);
    }
    printf("\nExecution time: %f seconds\n", cpu_time_used);
    return 0;
}
```

Output:


"C:\Users\cw\Desktop\4th ser
Enter low: 0
Enter mid: 2
Enter high: 5
Enter 6 sorted elements:
2 5 8 1 6 9
Merged array:
1 2 5 6 8 9
Execution time: 0.000000 seconds
Process returned 0 (0x0)   execution time : 11.327 s
Press any key to continue.

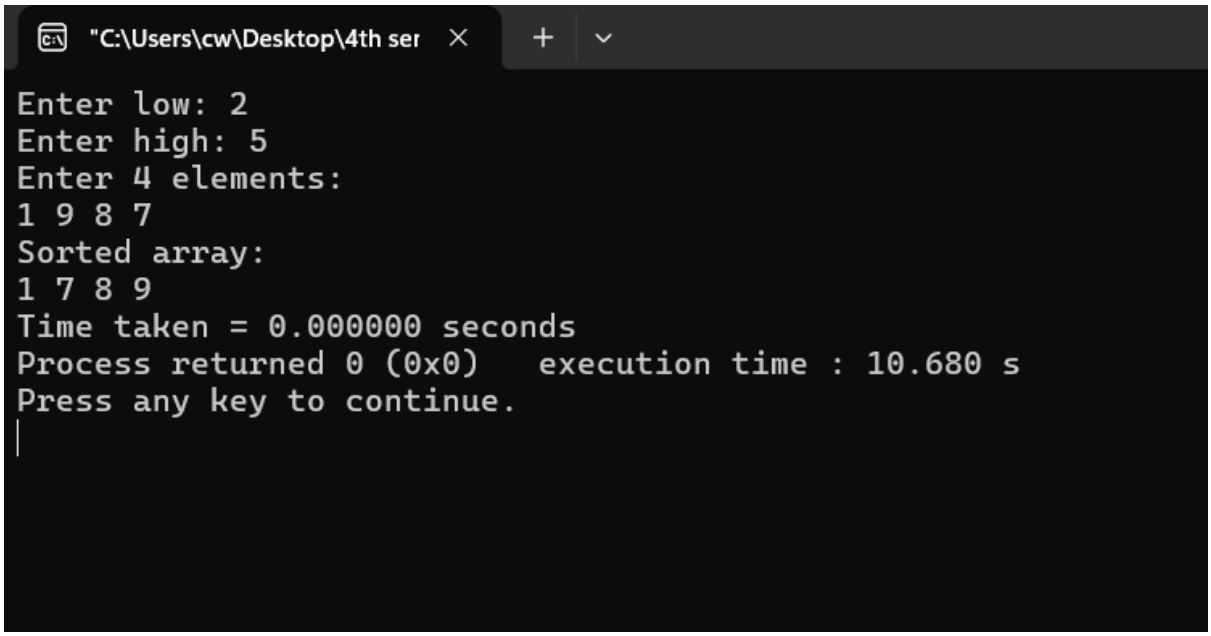
Lab Program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include <stdio.h>
#include <time.h>
int partition(int a[100], int low, int high){
   int i, j, key, temp;
   i = low + 1;
   j = high;
   key = a[low];
   while(1){
      while(a[i] <= key && i < high) i++;
      while(a[j] > key) j--;
      if(i < j){
         temp = a[i];
         a[i] = a[j];
         a[j] = temp;
      }
      else{
         temp = a[low];
         a[low] = a[j];
         a[j] = temp;
         return j;
      }
   }
}
void quicksort(int a[100], int low, int high){
   int j;
   if(low < high){
      j = partition(a, low, high);
```

```
        quicksort(a, low, j-1);
        quicksort(a, j+1, high);
    }
}
int main(){
    int low, high, i;
    clock_t start, end;
    double time_taken;
    printf("Enter low: ");
    scanf("%d",&low);
    printf("Enter high: ");
    scanf("%d",&high);
    int a[100];
    printf("Enter %d elements:\n",(high - low) + 1);
    for(i = low; i <= high; i++){
        scanf("%d",&a[i]);
    }
    start = clock();
    quicksort(a, low, high);
    end = clock();
    printf("Sorted array:\n");
    for(i = low; i <= high; i++){
        printf("%d ", a[i]);
    }
    time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("\nTime taken = %f seconds", time_taken);
    return 0;
}
```

Output:


"C:\Users\cw\Desktop\4th ser
Enter low: 2
Enter high: 5
Enter 4 elements:
1 9 8 7
Sorted array:
1 7 8 9
Time taken = 0.000000 seconds
Process returned 0 (0x0)   execution time : 10.680 s
Press any key to continue.

Lab program 5:

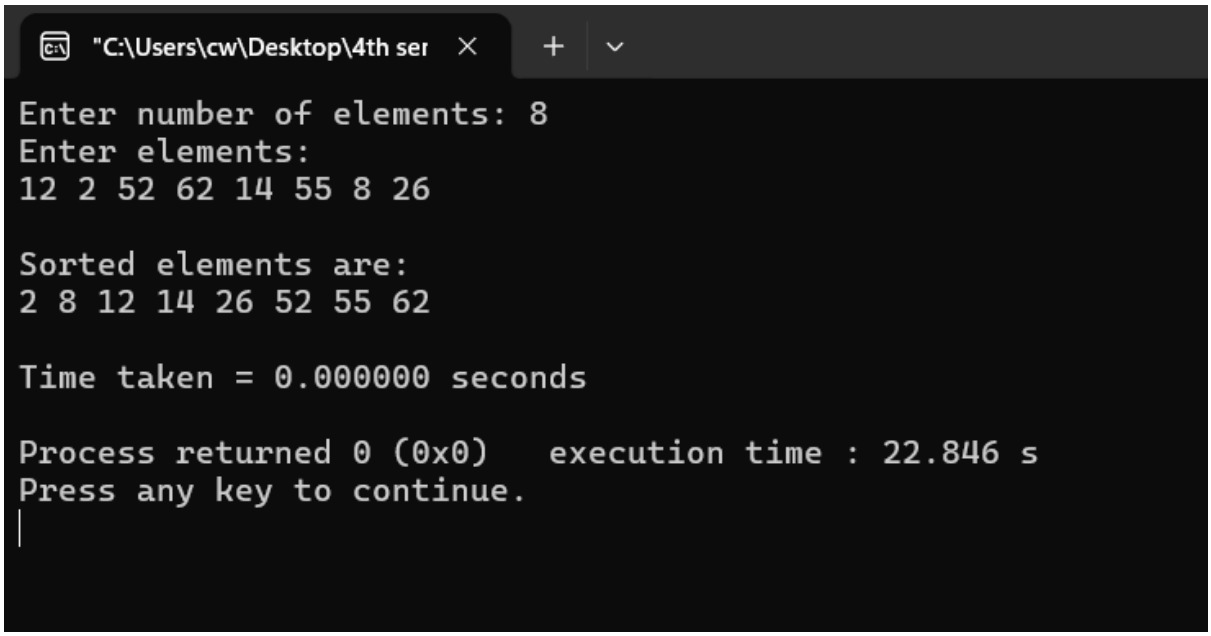
Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void heapify(int a[], int n, int i)
{
   int c, item;
   int p=i;
   item=a[p];
   c=2*p+1;
   while(c<=n-1){
      if(c+1<=n-1){
         if(a[c]<a[c+1]) c++;
      }
      if(item<a[c]){
         a[p]=a[c];
         p=c;
         c=2*p+1;
      }
      else break;
   }
   a[p]=item;
}
void buildheap(int a[],int n){
   int i;
   for(i=(n-1)/2;i>=0;i--){
      heapify(a,n,i);
```

```
    }
}
void heapSort(int a[], int n)
{
    int i, temp;
    buildheap(a,n);
    for(i=n-1;i>0;i--){
        temp=a[0];
        a[0]=a[i];
        a[i]=temp;
        heapify(a,i,0);
    }
}

int main()
{
    int n;
    clock_t start, end;
    double cpu_time;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    start = clock();
    heapSort(a, n);
    end = clock();
    cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;
```

```
    printf("\nSorted elements are:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n\nTime taken = %f seconds\n", cpu_time);
    return 0;
}
```

Output:



Lab Program 6:

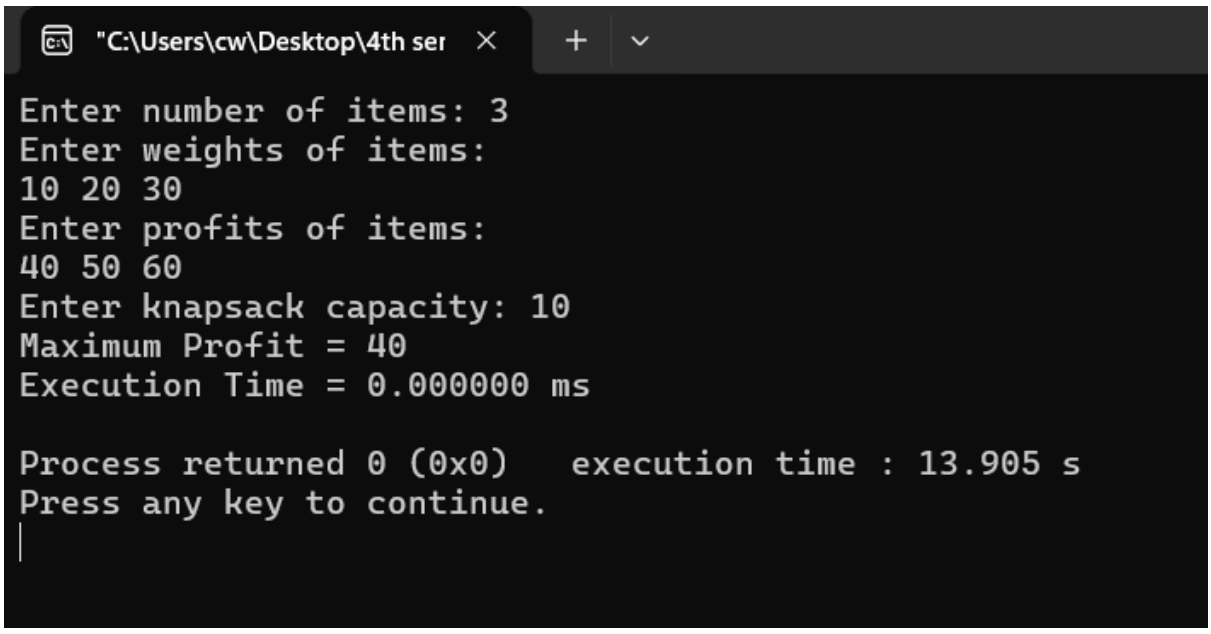
Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>
#include <time.h>

int max(int a, int b)
{
   if (a > b)
      return a;
   else
      return b;
}

int knapsackDP(int n, int M, int W[], int P[])
{
   int i, j;
   int Table[n + 1][M + 1];

   for (i = 0; i <= n; i++)
      Table[i][0] = 0;

   for (j = 0; j <= M; j++)
      Table[0][j] = 0;

   for (i = 1; i <= n; i++)
   {
      for (j = 1; j <= M; j++)
      {
         if (j < W[i - 1])
         {
            Table[i][j] = Table[i - 1][j];
         }
         else
```

```
            {
                Table[i][j] = max(
                    Table[i - 1][j],
                    P[i - 1] + Table[i - 1][j - W[i - 1]]
                );
            }
        }
    }

    return Table[n][M];
}

int main()
{
    int n, M;
    clock_t start, end;

    printf("Enter number of items: ");
    scanf("%d", &n);

    int W[n], P[n];

    printf("Enter weights of items:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &W[i]);

    printf("Enter profits of items:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &P[i]);

    printf("Enter knapsack capacity: ");
    scanf("%d", &M);

    start = clock();
```

```
    int result = knapsackDP(n, M, W, P);

    end = clock();

    printf("Maximum Profit = %d\n", result);

    double time_taken =
        ((double)(end - start) * 1000) / CLOCKS_PER_SEC;

    printf("Execution Time = %lf ms\n", time_taken);

    return 0;
}
```

Output:



Lab Program 7:

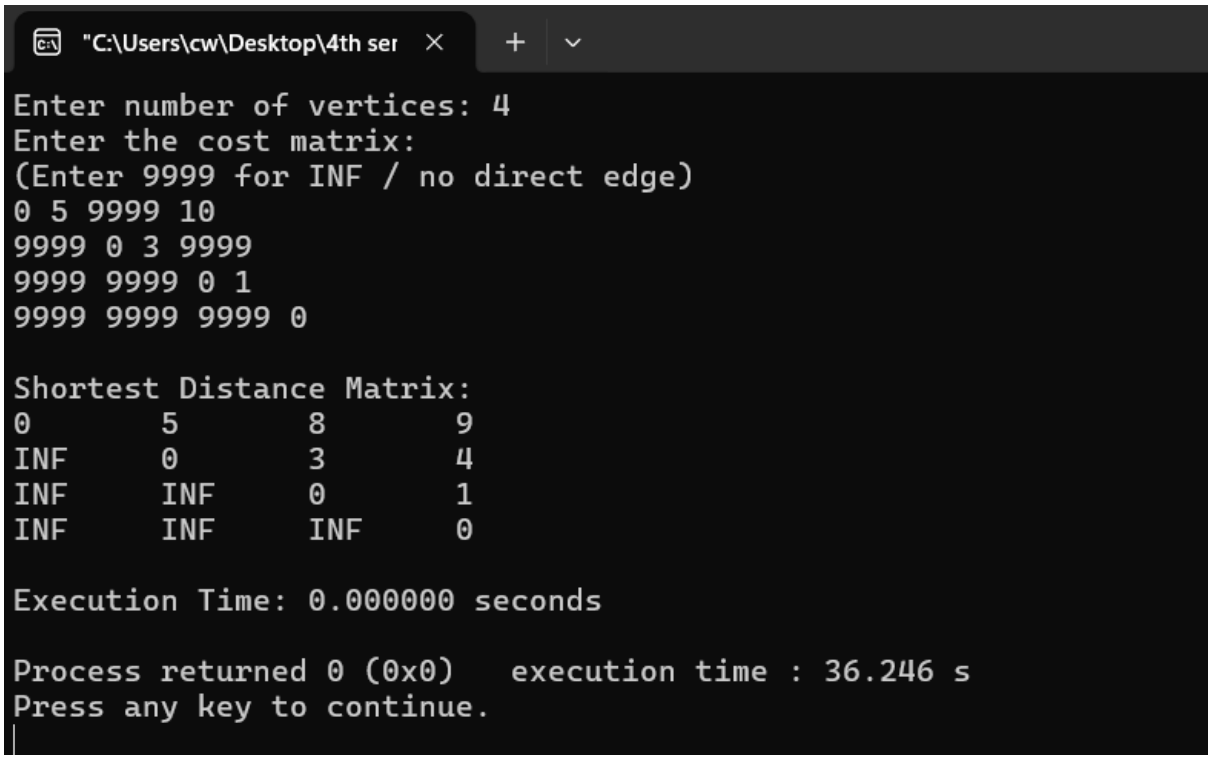
Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>
#include <time.h>
#define MAX 20
#define INF 9999
void floyds(int n, int cost[MAX][MAX], int A[MAX][MAX]) {
   int i, j, k;
   for(i = 0; i < n; i++) {
      for(j = 0; j < n; j++) {
         A[i][j] = cost[i][j];
      }
   }
   for(k = 0; k < n; k++) {
      for(i = 0; i < n; i++) {
         for(j = 0; j < n; j++) {
            if(A[i][k] + A[k][j] < A[i][j]) {
               A[i][j] = A[i][k] + A[k][j];
            }
         }
      }
   }
}
int main() {
   int n;
   int cost[MAX][MAX], A[MAX][MAX];
   int i, j;
```

```
clock_t start, end;
double cpu_time_used;
printf("Enter number of vertices: ");
scanf("%d", &n);
printf("Enter the cost matrix:\n");
printf("(Enter %d for INF / no direct edge)\n", INF);
for(i = 0; i < n; i++) {
   for(j = 0; j < n; j++) {
      scanf("%d", &cost[i][j]);
   }
}
start = clock();
floyds(n, cost, A);
end = clock();
cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("\nShortest Distance Matrix:\n");
for(i = 0; i < n; i++) {
   for(j = 0; j < n; j++) {
      if(A[i][j] == INF)
         printf("INF\t");
      else
         printf("%d\t", A[i][j]);
   }
   printf("\n");
}
printf("\nExecution Time: %f seconds\n", cpu_time_used);
return 0;
```

}

Output:


"C:\Users\cw\Desktop\4th ser
Enter number of vertices: 4
Enter the cost matrix:
(Enter 9999 for INF / no direct edge)
0 5 9999 10
9999 0 3 9999
9999 9999 0 1
9999 9999 9999 0
Shortest Distance Matrix:
0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0
Execution Time: 0.000000 seconds
Process returned 0 (0x0) execution time : 36.246 s
Press any key to continue.

Lab Program 8 (a):

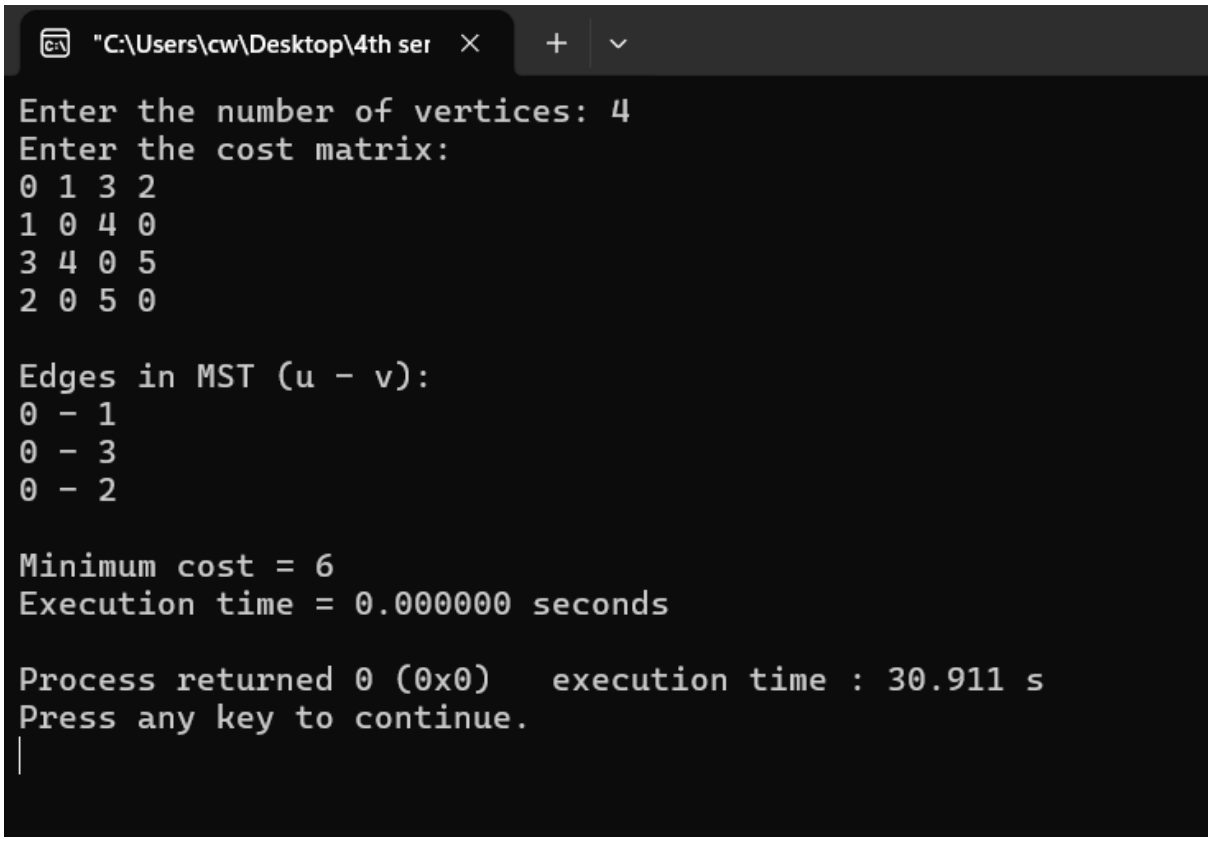
Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>
#include <time.h>
#define MAX 20
#define INF 9999
int prims(int n, int cost[MAX][MAX], int T[MAX][2]);

int main() {
    int n, cost[MAX][MAX], T[MAX][2];
    clock_t start, end;
    double cpu_time_used;
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    printf("Enter the cost matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0 && i != j)
                cost[i][j] = INF;
        }
    }
    start = clock();
    int mincost = prims(n, cost, T);
    end = clock();
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nEdges in MST (u - v):\n");
    for (int i = 0; i < n - 1; i++) {
```

```
        printf("%d - %d\n", T[i][0], T[i][1]);
    }
    printf("\nMinimum cost = %d\n", mincost);
    printf("Execution time = %f seconds\n", cpu_time_used);
    return 0;
}
int prims(int n, int cost[MAX][MAX], int T[MAX][2]) {
    int visited[MAX] = {0};
    int mincost = 0;
    int edges = 0;
    visited[0] = 1;
    while (edges < n - 1) {
        int min = INF;
        int u = -1, v = -1;
        for (int i = 0; i < n; i++) {
            if (visited[i]) {
                for (int j = 0; j < n; j++) {
                    if (!visited[j] && cost[i][j] < min) {
                        min = cost[i][j];
                        u = i;
                        v = j;
                    }
                }
            }
        }
        if (u != -1 && v != -1) {
            T[edges][0] = u;
            T[edges][1] = v;
```

```
            mincost += min;
            visited[v] = 1;
            edges++;
        }
    }
    return mincost;
}
```

OUTPUT:



Lab Program 8(b):

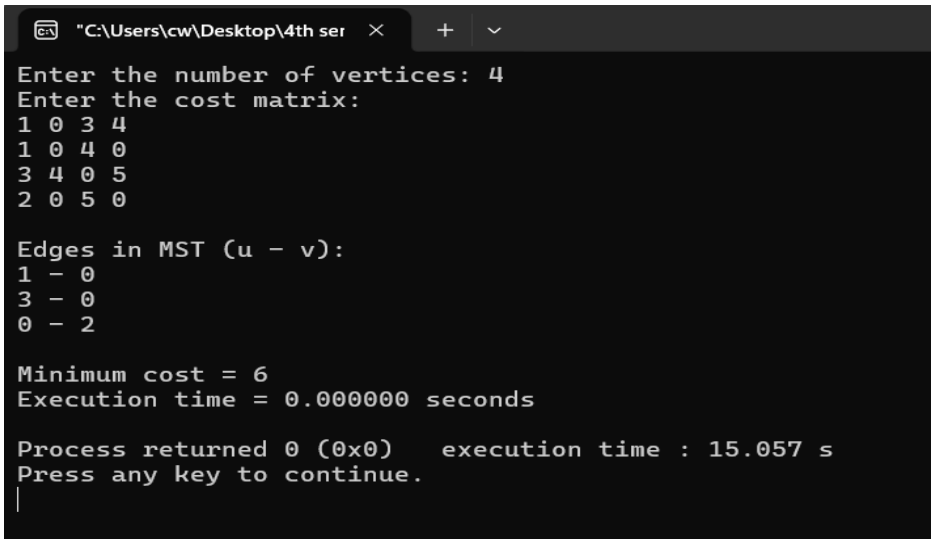
Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include <stdio.h>
#include <time.h>
#define MAX 20
#define INF 9999
int parent[MAX];
int find(int i) {
   while (parent[i] != i)
      i = parent[i];
   return i;
}
void unionSet(int i, int j) {
   int a = find(i);
   int b = find(j);
   parent[a] = b;
}
int kruskal(int n, int cost[MAX][MAX], int T[MAX][2]) {
   int mincost = 0;
   int edges = 0;
   for (int i = 0; i < n; i++)
      parent[i] = i;
   while (edges < n - 1) {
      int min = INF;
      int u = -1, v = -1;
      for (int i = 0; i < n; i++) {
```

```
            for (int j = 0; j < n; j++) {
                if (i != j && cost[i][j] < min) {
                    min = cost[i][j];
                    u = i;
                    v = j;
                }
            }
        }
        if (find(u) != find(v)) {
            T[edges][0] = u;
            T[edges][1] = v;
            mincost += min;
            unionSet(u, v);
            edges++;
        }
        cost[u][v] = cost[v][u] = INF;
    }
    return mincost;
}
int main() {
    int n, cost[MAX][MAX], T[MAX][2];
    clock_t start, end;
    double cpu_time_used;
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    printf("Enter the cost matrix:\n");
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0 && i != j)
                cost[i][j] = INF;
        }
    }
    start = clock();
    int mincost = kruskal(n, cost, T);
    end = clock();
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nEdges in MST (u - v):\n");
    for (int i = 0; i < n - 1; i++) {
        printf("%d - %d\n", T[i][0], T[i][1]);
    }
    printf("\nMinimum cost = %d\n", mincost);
    printf("Execution time = %f seconds\n", cpu_time_used);
    return 0;
}
```

Output:



Lab Program 9:

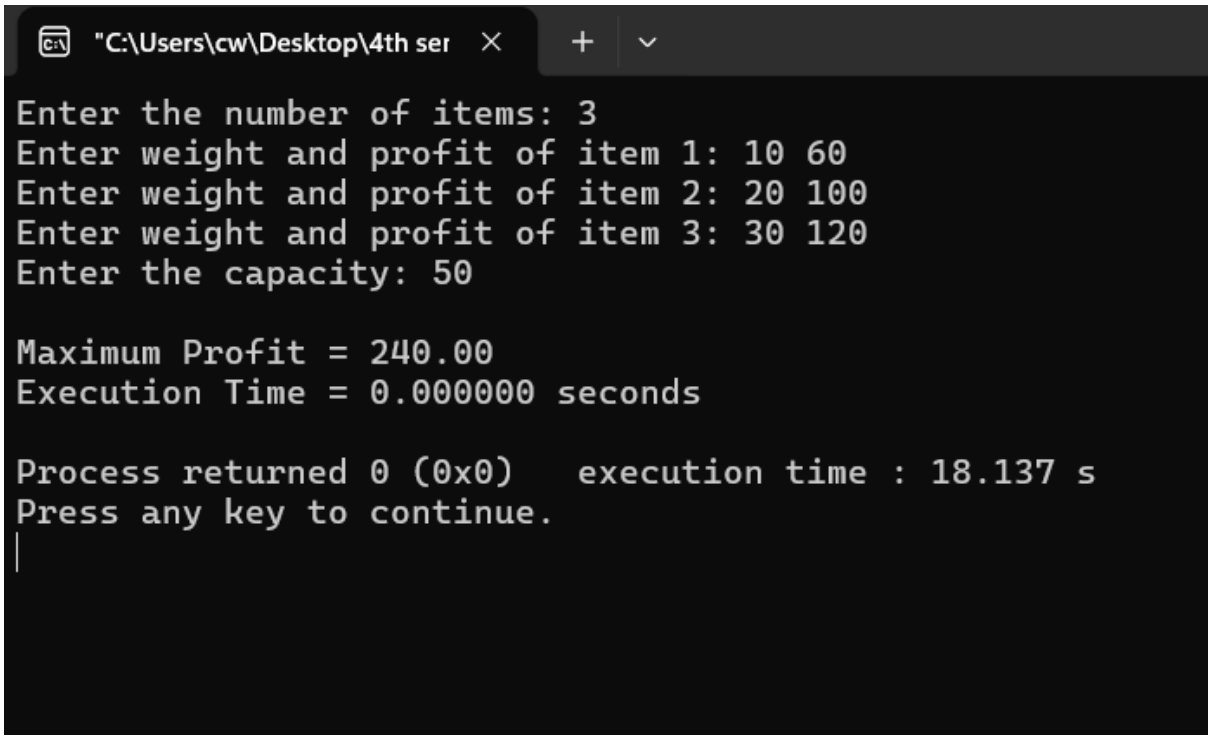
Implement Fractional Knapsack using Greedy technique.

```
#include <stdio.h>
#include <time.h>
float fractionalKnapsack(int remcap, int w[], int p[], float x[], int n)
{
   float maxprofit = 0.0;
   for (int i = 0; i < n; i++)
   {
      if (remcap >= w[i])
      {
         remcap = remcap - w[i];
         maxprofit = maxprofit + p[i];
      }
      else
      {
         maxprofit = maxprofit + x[i] * remcap;
         break;
      }
   }
   return maxprofit;
}
void sort(int w[], int p[], float x[], int n)
{
   for (int i = 0; i < n - 1; i++)
   {
      for (int j = i + 1; j < n; j++)
```

```
        {
            if (x[i] < x[j])
            {
                float temp = x[i];
                x[i] = x[j];
                x[j] = temp;
                int tempw = w[i];
                w[i] = w[j];
                w[j] = tempw;
                int tempp = p[i];
                p[i] = p[j];
                p[j] = tempp;
            }
        }
    }
}
int main()
{
    int n, remcap;
    clock_t start, end;
    printf("Enter the number of items: ");
    scanf("%d", &n);
    int w[n], p[n];
    float x[n];
    for (int i = 0; i < n; i++)
    {
        printf("Enter weight and profit of item %d: ", i + 1);
```

```
        scanf("%d %d", &w[i], &p[i]);
        x[i] = (float)p[i] / w[i];
    }
    sort(w, p, x, n);
    printf("Enter the capacity: ");
    scanf("%d", &remcap);
    start = clock();
    float maxp = fractionalKnapsack(remcap, w, p, x, n);
    end = clock();
    double time_taken =
        ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nMaximum Profit = %.2f\n", maxp);
    printf("Execution Time = %lf seconds\n", time_taken);
    return 0;
}
```

Output:



Lab program 10:

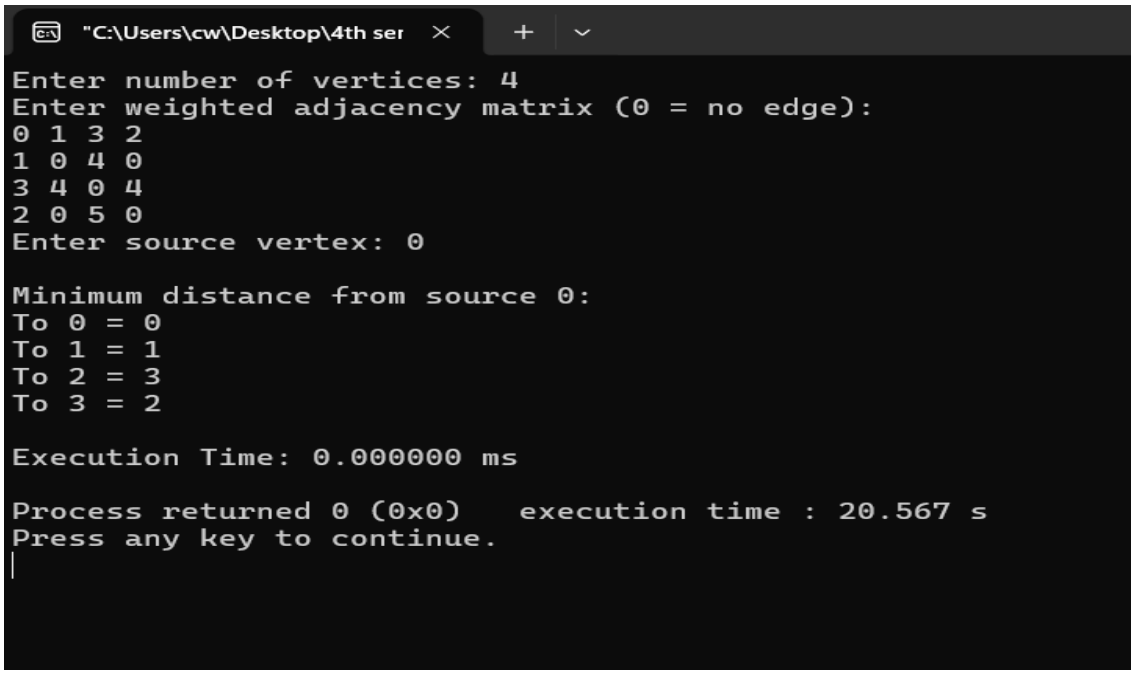
From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>
#include <time.h>
#define MAX 100
#define INF 99999
int minDistance(int dist[], int visited[], int n)
{
   int min = INF, min_index = -1;
   for (int i = 0; i < n; i++)
   {
      if (visited[i] == 0 && dist[i] < min)
      {
         min = dist[i];
         min_index = i;
      }
   }
   return min_index;
}
void dijkstra(int graph[MAX][MAX], int n, int src)
{
   int dist[MAX], visited[MAX];
   for (int i = 0; i < n; i++)
   {
      dist[i] = INF;
      visited[i] = 0;
```

```
    }
    dist[src] = 0;
    for (int count = 0; count < n - 1; count++)
    {
        int u = minDistance(dist, visited, n);
        visited[u] = 1;
        for (int v = 0; v < n; v++)
        {
            if (!visited[v] &&
                graph[u][v] &&
                dist[u] != INF &&
                dist[u] + graph[u][v] < dist[v])
            {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
    printf("\nMinimum distance from source %d:\n", src);
    for (int i = 0; i < n; i++)
    {
        printf("To %d = %d\n", i, dist[i]);
    }
}
int main()
{
    int n, graph[MAX][MAX], src;
    clock_t start, end;
```

```
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter weighted adjacency matrix (0 = no edge):\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }
    printf("Enter source vertex: ");
    scanf("%d", &src);
    start = clock();
    dijkstra(graph, n, src);
    end = clock();
    double time_taken =
        ((double)(end - start) * 1000) / CLOCKS_PER_SEC;
    printf("\nExecution Time: %lf ms\n", time_taken);
    return 0;
}
```

Output:


"C:\Users\cw\Desktop\4th ser
Enter number of vertices: 4
Enter weighted adjacency matrix (0 = no edge):
0 1 3 2
1 0 4 0
3 4 0 4
2 0 5 0
Enter source vertex: 0
Minimum distance from source 0:
To 0 = 0
To 1 = 1
To 2 = 3
To 3 = 2
Execution Time: 0.000000 ms
Process returned 0 (0x0) execution time : 20.567 s
Press any key to continue.

Lab Program 11:

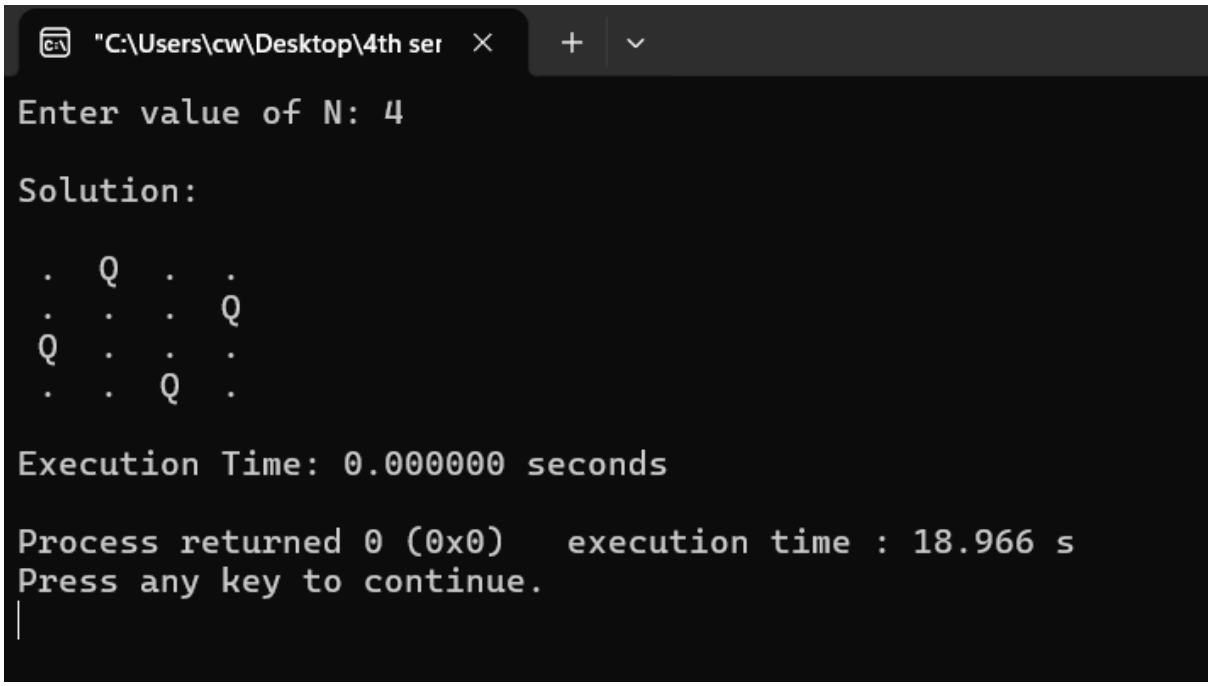
Implement "N-Queens Problem" using Backtracking.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 20
int board[MAX][MAX];
int N;
void printBoard() {
   int i, j;
   printf("\nSolution:\n\n");
   for(i = 0; i < N; i++) {
      for(j = 0; j < N; j++) {
         if(board[i][j] == 1)
            printf(" Q ");
         else
            printf(" . ");
      }
      printf("\n");
   }
}
int isSafe(int row, int col) {
   int i, j;
   for(i = 0; i < row; i++) {
      if(board[i][col] == 1)
         return 0;
   }
```

```
    for(i = row - 1, j = col - 1;
        i >= 0 && j >= 0;
        i--, j--) {
        if(board[i][j] == 1)
            return 0;
    }
    for(i = row - 1, j = col + 1;
        i >= 0 && j < N;
        i--, j++) {
        if(board[i][j] == 1)
            return 0;
    }
    return 1;
}
int solveNQueens(int row) {
    if(row == N)
        return 1;
    int col;
    for(col = 0; col < N; col++) {
        if(isSafe(row, col)) {
            board[row][col] = 1;
            if(solveNQueens(row + 1))
                return 1;
            board[row][col] = 0;
        }
    }
    return 0;
```

```
}
int main() {
    int i, j;
    printf("Enter value of N: ");
    scanf("%d", &N);
    for(i = 0; i < N; i++) {
        for(j = 0; j < N; j++) {
            board[i][j] = 0;
        }
    }
    clock_t start, end;
    start = clock();
    if(solveNQueens(0)) {
        printBoard();
    }
    else {
        printf("No solution exists.\n");
    }
    end = clock();
    double time_taken =
        ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nExecution Time: %f seconds\n", time_taken);
    return 0;
}
```

Output:


"C:\Users\cw\Desktop\4th ser
Enter value of N: 4
Solution:
. Q . .
. . . Q
Q . . .
. . Q .
Execution Time: 0.000000 seconds
Process returned 0 (0x0) execution time : 18.966 s
Press any key to continue.

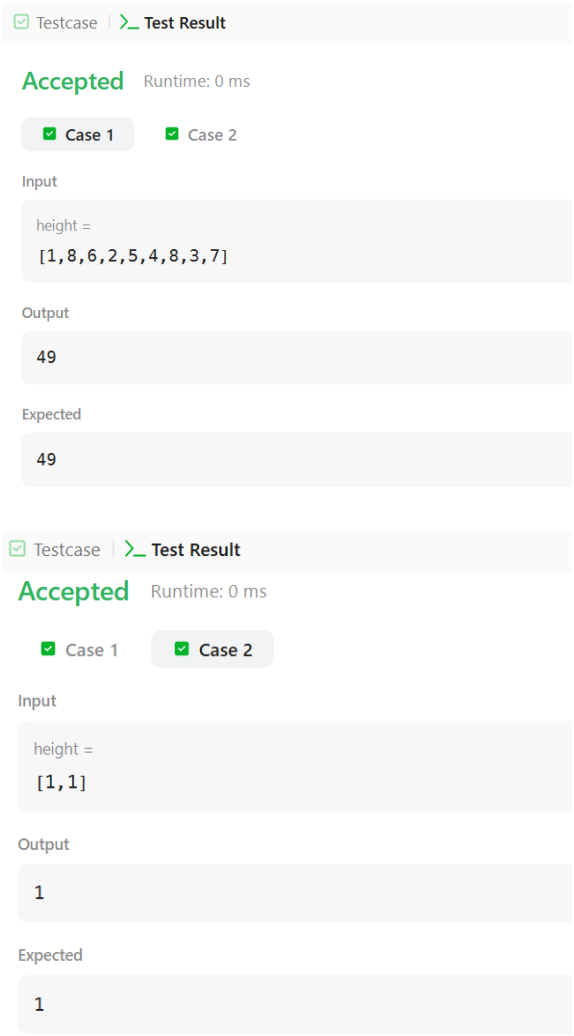
LEETCODE PROBLEMS:

11.Container with more water

```
int maxArea(int height[], int heightSize)
{
    int left = 0;
    int right = heightSize - 1;
    int max = 0;
    while (left < right)
    {
        int width = right - left;
        int h;
        if (height[left] < height[right])
        {
            h = height[left];
            left++;
        }
        else
        {
            h = height[right];
            right--;
        }
        int area = h * width;
        if (area > max)
        {
            max = area;
        }
    }
```

```
    return max;
}
```

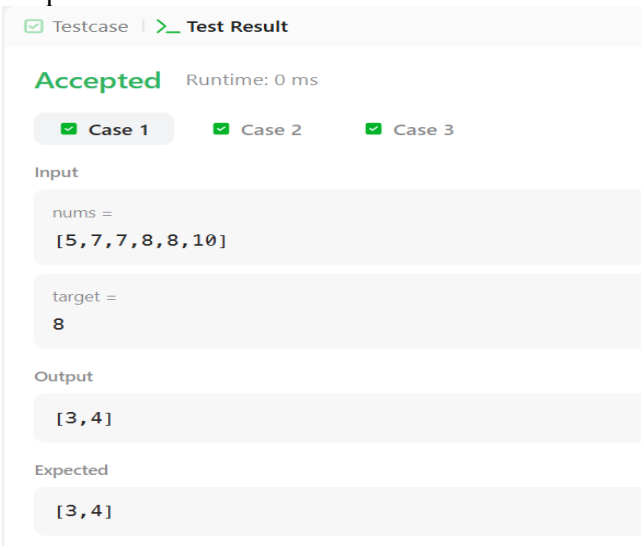
Output:

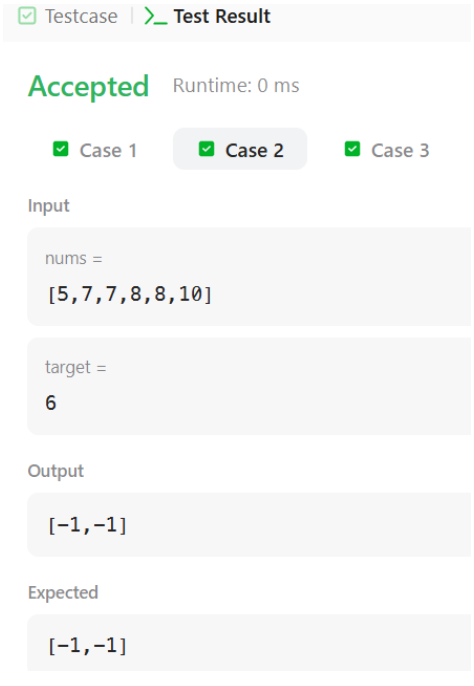


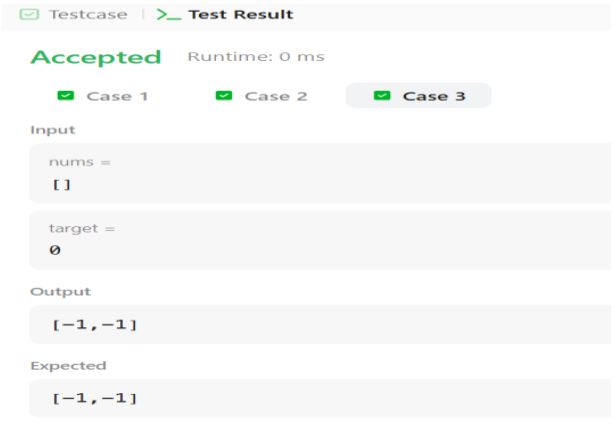
34.FIND FIRST AND LAST POSITION OF AN ELEMENT IN SORTED ARRAY

```
int* searchRange(int* nums, int numsSize, int target, int* returnSize)
{
    int *result = (int*)malloc(2 * sizeof(int));
    *returnSize = 2;

    int start = -1, end = -1;

    for (int i = 0; i < numsSize; i++)
    {
        if (nums[i] == target)
        {
            if (start == -1)
                start = i;

            end = i;
        }
    }

    result[0] = start;
    result[1] = end;

    return result;
}
```

Output:



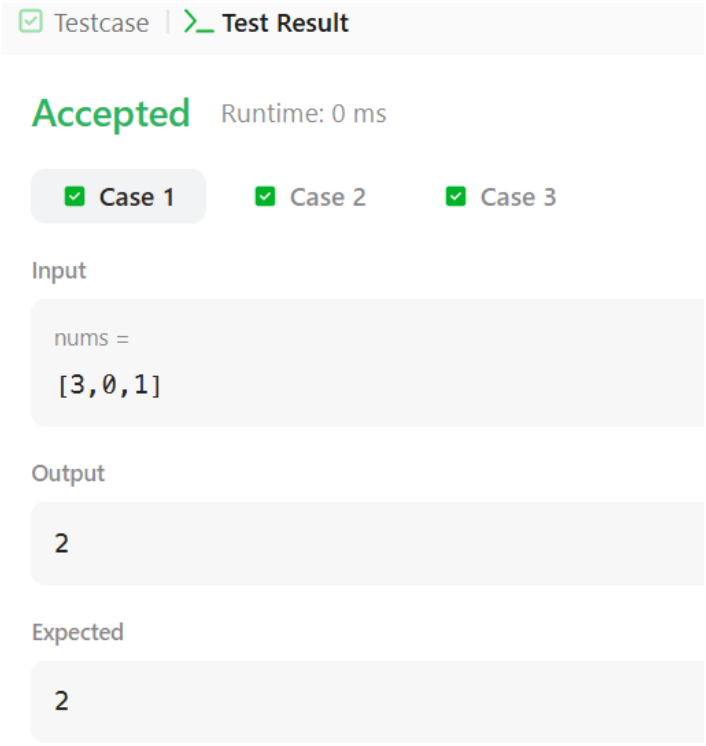




268: MISSING NUMBER

```
int missingNumber(int* nums, int numsSize) {
    int sum1=0,n,i,sum2=0;
    n=numsSize;
    for(int i=0;i<=numsSize;i++)
    {
      sum1=sum1+i;
    }
    for(int i=0;i<n;i++)
    {
      sum2=sum2+nums[i];
    }
    return sum1-sum2;
}
```

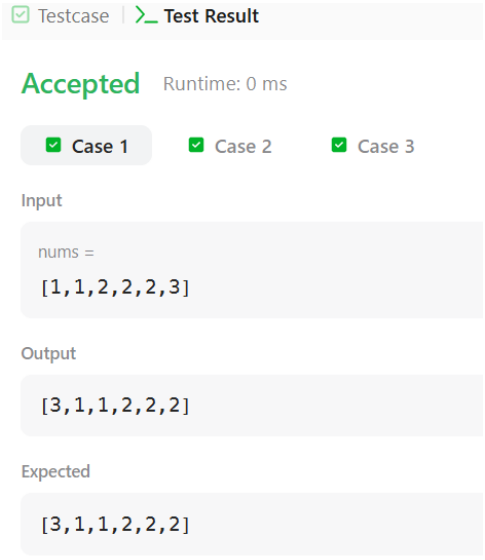
OUTPUT:

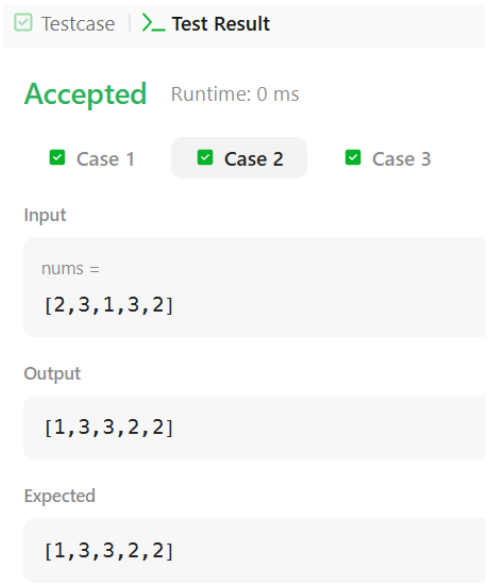


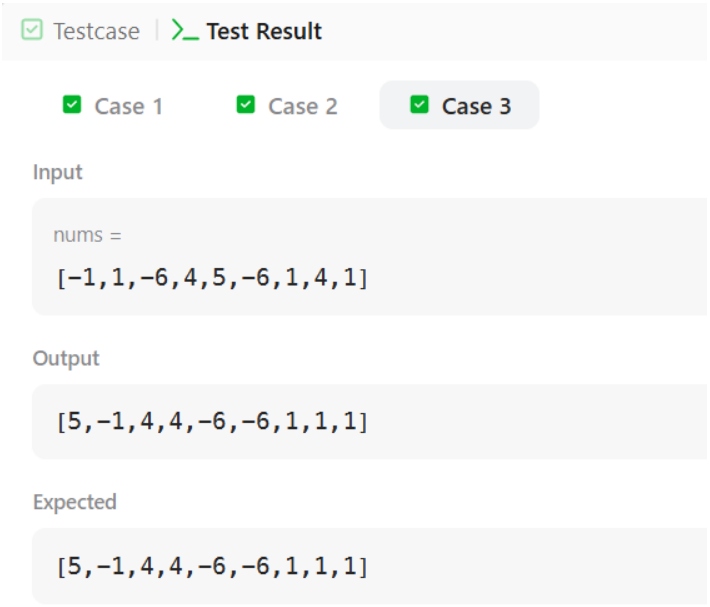
1636.SORT ARRAY BY INCREASING FREQUENCY

```
int* frequencySort(int* nums, int numsSize, int* returnSize)
{
    *returnSize = numsSize;
    int *result = (int *)malloc(numsSize * sizeof(int));
    int freq[201] = {0};
    for (int i = 0; i < numsSize; i++)
    {
        freq[nums[i] + 100]++;
        result[i] = nums[i];
    }
    for (int i = 0; i < numsSize - 1; i++)
    {
        for (int j = i + 1; j < numsSize; j++)
        {
            int fi = freq[result[i] + 100];
            int fj = freq[result[j] + 100];
            if (fi > fj || (fi == fj && result[i] < result[j]))
            {
                int temp = result[i];
                result[i] = result[j];
                result[j] = temp;
            }
        }
    }
    return result;
}
```

Output:


Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[1,1,2,2,2,3]
Output
[3,1,1,2,2,2]
Expected
[3,1,1,2,2,2]


Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[2,3,1,3,2]
Output
[1,3,3,2,2]
Expected
[1,3,3,2,2]

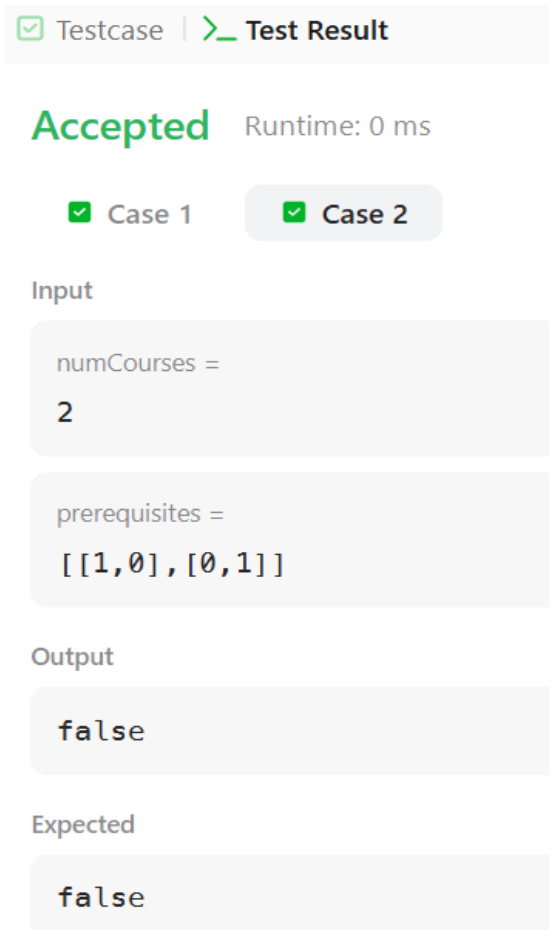

Testcase
Test Result
Case 1
Case 2
Case 3
Input
nums =
[-1,1,-6,4,5,-6,1,4,1]
Output
[5,-1,4,4,-6,-6,1,1,1]
Expected
[5,-1,4,4,-6,-6,1,1,1]

207.COURSE SCHEDULE

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {
    int** graph = (int**)malloc(numCourses * sizeof(int*));
    int* indegree = (int*)calloc(numCourses, sizeof(int));
    int* graphColSize = (int*)calloc(numCourses, sizeof(int));
    for (int i = 0; i < numCourses; i++) {
        graph[i] = (int*)malloc(prerequisitesSize * sizeof(int));
    }
    for (int i = 0; i < prerequisitesSize; i++) {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];
        graph[b][graphColSize[b]++] = a;
        indegree[a]++;
    }
    int* queue = (int*)malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) {
            queue[rear++] = i;
        }
    }
    int count = 0;
    while (front < rear) {
        int curr = queue[front++];
        count++;
        for (int i = 0; i < graphColSize[curr]; i++) {
```

```
            int neighbor = graph[curr][i];
            indegree[neighbor]--;
            if (indegree[neighbor] == 0) {
                queue[rear++] = neighbor;
            }
        }
    }
    return count == numCourses;
}
```

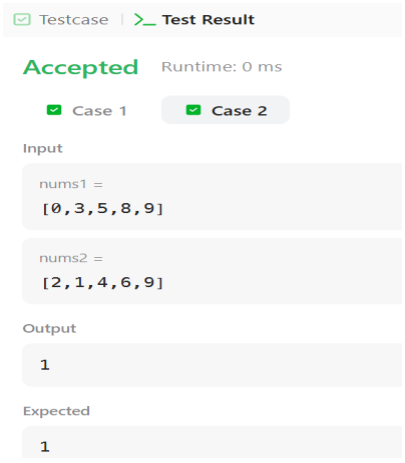
OUTPUT:



801.MINIMUM SWAPS TO MAKE SEQUENCES INCREASING

```
int minSwap(int* nums1, int n, int* nums2, int m) {
    int keep = 0;
    int swap = 1;
    for (int i = 1; i < n; i++) {
        int newkeep = 999999;
        int newswap = 999999;
        if (nums1[i] > nums1[i - 1] && nums2[i] > nums2[i - 1]) {
            newkeep = keep;
            newswap = swap + 1;
        }
        if (nums1[i] > nums2[i - 1] && nums2[i] > nums1[i - 1]) {
            if (swap < newkeep)
                newkeep = swap;
            if (keep + 1 < newswap)
                newswap = keep + 1;
        }
        keep = newkeep;
        swap = newswap;
    }
    return (keep < swap) ? keep : swap;
}
```

OUTPUT:



BHV

287.FINDING THE DUPLICATE NUMBER

```
int findDuplicate(int* nums, int n) {
    int* seen = calloc(n, sizeof(int));
    for (int i = 0; i < n; i++) {
        if (seen[nums[i]]) {
            free(seen);
            return nums[i];
        }
        seen[nums[i]] = 1;
    }
    free(seen);
    return -1;
}
```

OUTPUT: